MarketSys Interface Protocol Specification

Review proposal - October 24, 2026. This document defines the proposed app-to-controller interface. Firmware does not yet implement this host protocol. Session versus persistent event identity remains an engineer decision. The CRC procedure below is exact for the current draft; changing frame fields requires regenerating the frame examples, not changing the CRC algorithm.

1. Application boundary

The app reports one head's destination as that head enters the penning alley after sale. Destination is the only business payload. The controller derives ACTIVE, LAST, HOLD_OPEN and OFF. A new event and a retransmission must have different delivery semantics. Buyer/lot/animal identification and indicator state codes are not required in this outgoing message.

2. Fixed-size outgoing HEAD_DESTINATION frame

With the proposed session and transaction fields, the outgoing HEAD_DESTINATION frame is 16 bytes:

Zero-based offset	COBOL position	Size	Field	Current draft
0-1	1-2	2	SYNC	AA 55
2	3	1	VERSION	02
3	4	1	TYPE	10 = HEAD_DESTINATION
4-7	5-8	4	SESSION_ID	Proposed event-stream identity; review pending
8-11	9-12	4	TRANSACTION_ID	Proposed per-head event number
12	13	1	DESTINATION_NUMBER	Numeric pen number, up to 255
13	14	1	ALPHA_DESIGNATION	Reserved for letter designation
14-15	15-16	2	CRC16	Low byte, then high byte

The two destination bytes encode a pen number and an optional alpha designation. The worked examples use 00 for no letter and ASCII B (42 hex) for B; that alpha encoding/valid range and whether number 0 is reserved still need final agreement.

Session/transaction integers use little-endian byte order in this draft. If the engineer selects another event-identity structure, update offsets, frame length and CRC test vectors before implementation. The session field is proposed and remains subject to review.

3. Exact CRC definition

Use CRC-16/CCITT-FALSE, also catalogued as CRC-16/IBM-3740:

Parameter	Value
Width	16 bits
Polynomial	1021 hex = 4129 decimal
Initial register	FFFF hex = 65535 decimal, for EACH frame
Input reflected	No
Output reflected	No
Final XOR	0000; no final alteration
Check for ASCII bytes 31..39 (123456789)	29B1 hex
Wire order of CRC	Low byte first, high byte second

The algorithm parameters and alias are documented in the CRC RevEng catalogue. A routine named only CRC16 or CRC-CCITT is not sufficient: its initial value, reflection and final-XOR settings must match this table.

3.1 Exactly which bytes COBOL processes

First construct the raw outgoing frame, with its final header, IDs and destination. Calculate over VERSION through ALPHA_DESIGNATION, in wire order:

```
Excluded     Included in CRC, exactly 12 bytes              Excluded
AA 55     | 02 10 78 56 34 12 03 00 00 00 14 00           | CRC CRC
SYNC        VERSION ... ALPHA                              appended later
```

For the 16-byte draft, this is COBOL `FRAME(3:12)`. It means start at position 3 and take 12 bytes; the second number is a length, not an end index. Do not include SYNC, the CRC placeholders, unused buffer padding, spaces, serial delimiters, CR/LF or extra zero bytes. Zero-valued fields inside the 12 covered bytes ARE included and each goes through all eight bit steps.

The printed text `02 10` represents two raw bytes, 02 and 10 hex. Do not calculate over the ASCII characters '0', '2', space, '1', '0'. Likewise, destination decimal 20 is one raw byte 14 hex, not the characters "20".

3.2 Byte-by-byte calculation in COBOL-friendly arithmetic

The CRC register is a NUMBER between 0 and 65535, not a character string. Use an integer work field wide enough for at least 131070 during the shift. The tested example uses unsigned `PIC 9(9) COMP-5` work fields; it explicitly extracts bytes instead of depending on the native storage byte order.

For each covered input byte, from left to right:

1. Obtain its numeric value, 0..255.
2. Multiply that value by 256 and XOR it with the current CRC. This mixes the input byte into the HIGH byte of the register.
3. Repeat the following exactly eight times:
   - Save whether the OLD register is at least 32768 (its top bit is 1).
   - Multiply the register by 2 (one-bit left shift).
   - Keep only the remainder after dividing by 65536. This discards carry beyond bit 15 and keeps the register in the unsigned 16-bit range.
   - If the saved top bit was 1, XOR the register with 4129 (1021 hex).

4. Keep that result as the starting register for the next input byte.

Start with 65535 before the FIRST byte of EVERY frame. After the final byte, use the register directly: no complement, no final XOR, no additional shifts and no extra two zero bytes. Although integers and the CRC are transmitted low-byte first, this CRC calculation processes bits most-significant first. Do not reverse the bits within each incoming byte.

Equivalent pseudocode (XOR here means bitwise exclusive OR):

```
CRC = 65535
FOR EACH RAW BYTE B:
    CRC = CRC XOR (B * 256)
    REPEAT 8 TIMES:
        OLD_TOP_BIT = (CRC >= 32768)
        CRC = REMAINDER(CRC * 2, 65536)
        IF OLD_TOP_BIT:
            CRC = CRC XOR 4129
RETURN CRC
```

3.3 XOR without a vendor-specific COBOL bitwise function

XOR sets a result bit when the two input bits differ:

Input A bit	Input B bit	Result bit
0	0	0
0	1	1
1	0	1
1	1	0

XOR is not addition or ordinary OR. The tested `xor-16` paragraph in the COBOL source uses DIVIDE/REMAINDER to extract the low bit from each operand, adds its weight if the bits differ, then proceeds to the next bit. It repeats 16 times and requires no bitwise intrinsic or external CRC library.

```
xor-16.
    move 0 to ws-xor-result
    move 1 to ws-weight
    perform varying ws-xor-index from 1 by 1 until ws-xor-index > 16
        divide ws-xa by 2 giving ws-xa remainder ws-ra
        divide ws-xb by 2 giving ws-xb remainder ws-rb
        if ws-ra not = ws-rb add ws-weight to ws-xor-result end-if
        multiply 2 by ws-weight
    end-perform.
```

The routine consumes its two operand fields. See the complete compiled COBOL reference program for declarations, the enclosing eight-bit CRC loop and self-tests. It is a standalone reference harness, not a drop-in communications application or a callable API definition.

3.4 Obtaining and writing raw bytes with ORD and CHAR

COBOL ORD returns a 1-based ordinal position in the program's collating sequence. With the byte mapping assumed and tested by this reference:

```
compute ws-byte-value = function ord(ws-data(ws-byte-index:1)) - 1
```

Subtract 1: raw 00 must become numeric 0, not 1. The reverse operation is CHAR(value + 1), not CHAR(value). These rules follow the IBM ORD documentation and GnuCOBOL intrinsic-function reference. They refer to the program collating sequence, not a universally guaranteed raw-byte mapping under arbitrary compiler settings. The reference first tests ORD/CHAR mappings for 00, 80 and FF and stops if they do not match.

The engineer must validate those tests with the actual compiler/runtime and avoid automatic character-set conversion of the wire buffer. In particular, the standard check vector uses raw ASCII hex 313233343536373839; on an EBCDIC

platform the ordinary text literal "123456789" need not contain those bytes. Session IDs, transaction IDs and pen numbers must already be serialized to the agreed wire bytes BEFORE calculating the CRC.

3.5 Appending the CRC to the outgoing COBOL buffer

Split the final numeric CRC explicitly:

```
divide ws-crc by 256 giving ws-high remainder ws-low
compute ws-char-position = ws-low + 1
move function char(ws-char-position) to ws-frame(15:1)
compute ws-char-position = ws-high + 1
move function char(ws-char-position) to ws-frame(16:1)
```

For CRC 0B35 hex (2869 decimal), the quotient is 11 (0B hex) and remainder is 53 (35 hex). Therefore position 15 receives raw byte 35 and position 16 raw byte 0B. Transmit the resulting 16 raw bytes using the application's binary serial write operation. No particular COBOL communications API is assumed.

Do not MOVE a decimal DISPLAY field into the last two bytes; do not transmit the four printable characters "0B35"; and do not MOVE the native COMP-5 storage representation directly to the wire. Only the explicit two-byte serialization above defines the required byte order. The printable hex dump in the reference program is a diagnostic, not the transmit buffer.

4. Complete worked frame and intermediate results

Example: session 12345678 hex, transaction 3, pen number 20, no alpha (00):

```
AA 55 | 02 | 10 | 78 56 34 12 | 03 00 00 00 | 14 | 00 | 35 0B
SYNC    VER  TYPE SESSION       TRANSACTION    PEN  ALPHA CRC
```

CRC input and result after each complete byte:

Byte number	COBOL frame position	Input hex	CRC after 8 bit steps
Initial	-	-	FFFF
1	3	02	C1B2
2	4	10	695C
3	5	78	5E10
4	6	56	9108
5	7	34	ED4F
6	8	12	51F0
7	9	03	8AB7
8	10	00	87C2
9	11	00	236F
10	12	00	7B01
11	13	14	9C49
12	14	00	0B35

To debug the first byte, FFFF XOR 0200 = FDFF. Its eight subsequent register values are EBDF, C79F, 9F1F, 2E1F, 5C3E, B87C, 60D9, C1B2. This provides a checkpoint before comparing the remainder of the frame.

5. Receiver check and conformance tests

The receiver repeats the same calculation over positions 3..14 and compares the numeric result with `BYTE(15) + 256 * BYTE(16)`. Accept only an exact match, along with a supported version/type and valid field values. Do not advance flow on CRC failure. Do not assume running this algorithm over the whole little-endian-CRC frame must produce zero; this specification uses direct recomputation and comparison.

Raw CRC input	Expected numeric CRC hex	Low/high CRC bytes
Empty input (algorithm test only)	FFFF	FF FF
ASCII 123456789 = 31 32 33 34 35 36 37 38 39	29B1	B1 29
02 10 78 56 34 12 03 00 00 00 14 00	0B35	35 0B
Same example ending 14 42	63B3	B3 63
Every raw byte 00 through FF in ascending order	3FBD	BD 3F

The full reference COBOL program compiled and ran on Lin using isolated GnuCOBOL 3.2, without installing system packages. All five vectors, ORD/CHAR preflight and complete frame serialization passed. Independent Python bitwise calculation and Python binascii.crc_hqx agreed. Source: `examples/cobol/crc16_reference.cob`; independent checker: `scripts/crc_reference.py`. The engineer should compile/run the same vectors with the actual application compiler before integration; no test of that unprovided compiler or the app's serial API is claimed.

6. Remaining interface decisions

See engineer questions for persistent event counter versus session-plus-counter identity. CRC validity does not identify a retry; event identity does. Reply framing must distinguish controller acceptance from node-confirmed application/failure and correlate the event. Replies, resynchronization/timeouts, reset recovery and field allocations remain under review. AA55 can occur inside a frame: use the known version/type frame size and CRC, not a second AA55 as an automatic terminator.